Decode Attention의 Arithmetic Intensity 한계와 극복

핵심 질문: Transformer decode 단계에서 attention의 arithmetic intensity(AI)가 group size G라는 아키텍처 상수에 의해 결정되어 확장이 불가능한 문제를, 2024~2026년 사이에 어떻게 해결했는가?

목차

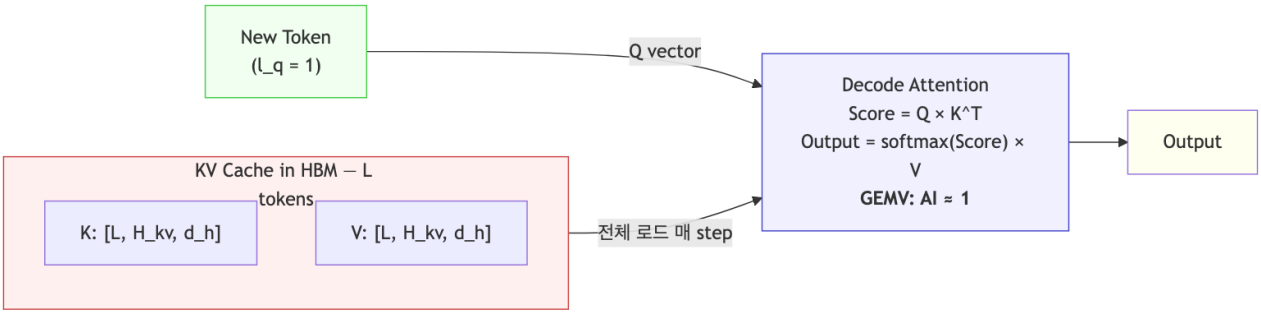
1. 문제 정의: 왜 Decode Attention의 AI는 상수인가
2. Group Size G와 AI의 수학적 관계
3. GPU 세대별 Ridge Point와 G의 격차
4. 해결책 1: MLA — Group Size 한계의 근본적 돌파
5. 해결책 2: 유효 Query 길이를 늘리는 접근
6. 해결책 3: KV Tying과 GTA
7. 해결책 4: 양자화로 AI 배수 증가
8. 해결책 5: Cross-Layer KV 공유
9. 종합 비교와 Roofline 위의 위치
10. 결론: 병목의 이동

1. 문제 정의: 왜 Decode Attention의 AI는 상수인가

1.1 Decode 단계의 특수성

Autoregressive decode에서 매 step마다:

- Query: 새로 생성된 토큰 1개 → l_q = 1
- Key/Value: 이전까지 누적된 KV cache 전체 → L 토큰 (수천~수만)



핵심: 단 1개의 query 벡터가 거대한 KV cache 전체를 읽어야 한다. 이는 본질적으로 GEMV(General Matrix-Vector multiply)이며, GEMV는 데이터 재활용이 불가능한 대표적인 memory-bound 연산이다.

1.2 FlashInfer의 Operational Intensity 공식

FlashInfer에서 도출한 attention의 operational intensity(OI) 일반 공식:

```
OI(attention) = O( 1 / (1/l_q + 1/l_kv) )
```

Phase	l_q	l_kv	OI	해석
Prefill	= L	= L	O(L)	L이 클수록 compute-bound
Decode	= 1	≫ 1	O(1)	항상 memory-bound
Chunked prefill	C	≫ C	O(C)	chunk 크기로 조절 가능

Decode에서 l_q = 1을 대입하면 l_kv가 아무리 커도 OI → O(1)로 수렴한다. Sequence length를 늘려도, batch size를 키워도 이 값은 변하지 않는다.

1.3 왜 Batching도 소용없는가

Linear layer(FC)에서는 batching이 AI를 올린다: B개의 request가 동일한 weight matrix를 공유하므로 weight 로드 비용이 B로 amortize된다.

```
AI_linear = O(B × d / (B + d)) → B가 클수록 AI 증가
```

그러나 attention에서는 각 request가 고유한 KV cache를 가진다. Request i의 KV cache는 request j와 공유할 수 없다:

```
Batch attention: 각 request 독립적으로 자신의 KV cache를 로드
→ 총 FLOPs = B × (단일 request FLOPs)
→ 총 Bytes = B × (단일 request Bytes)
→ AI = 변화 없음
```

> FlashInfer: "The operational intensity of attention kernels is irrelevant to batch size."

2. Group Size G와 AI의 수학적 관계

2.1 표기법

기호	의미	예시 (Llama-3 70B)
H_q	Query head 수	64
H_kv	KV head 수	8
G = H_q / H_kv	Group size	8
d_h	Head dimension	128
L	KV sequence length	4096
l_q	Query length (decode = 1)	1

2.2 Decode Attention AI 유도

FLOPs 계산 (단일 request, 전체 head):

각 query head h에 대해:

- Score: q_h^T × K_h → 2 × L × d_h FLOPs (L번의 내적, 각 d_h 차원)
- Context: attn_weights × V_h → 2 × L × d_h FLOPs

전체 H_q개 head 합산:

$$FLOPs_total = H_q \times (2 \times L \times d_h + 2 \times L \times d_h) = 4 \times H_q \times L \times d_h$$

Bytes 계산 (BF16 기준, 2 bytes/element):

항목	크기 (bytes)	비고
Q 로드	$H_q \times d_h \times 2$	작음, 무시 가능
K cache 로드	$H_kv \times L \times d_h \times 2$	지배적
V cache 로드	$H_kv \times L \times d_h \times 2$	지배적
Output 쓰기	$H_q \times d_h \times 2$	작음, 무시 가능

$L \gg 1$일 때 KV cache가 지배적:

$$Bytes_total \approx 2 \times H_kv \times L \times d_h \times 2 = 4 \times H_kv \times L \times d_h$$

Arithmetic Intensity:

$$AI = \frac{4 \times H_q \times L \times d_h}{4 \times H_kv \times L \times d_h} = \frac{H_q}{H_kv} = G$$

2.3 정밀 공식 (Lei Mao의 유도)

Q 로드와 output 쓰기를 포함한 정밀 공식:

$$AI_GQA = \frac{H_q \times L}{H_q + (H_q / G) \times L}$$

$$= \frac{1}{1/(G \times l_q) + 1/L}$$

Decode에서 $l_q = 1$, $L \gg G$:

$$AI_decode \approx G$$

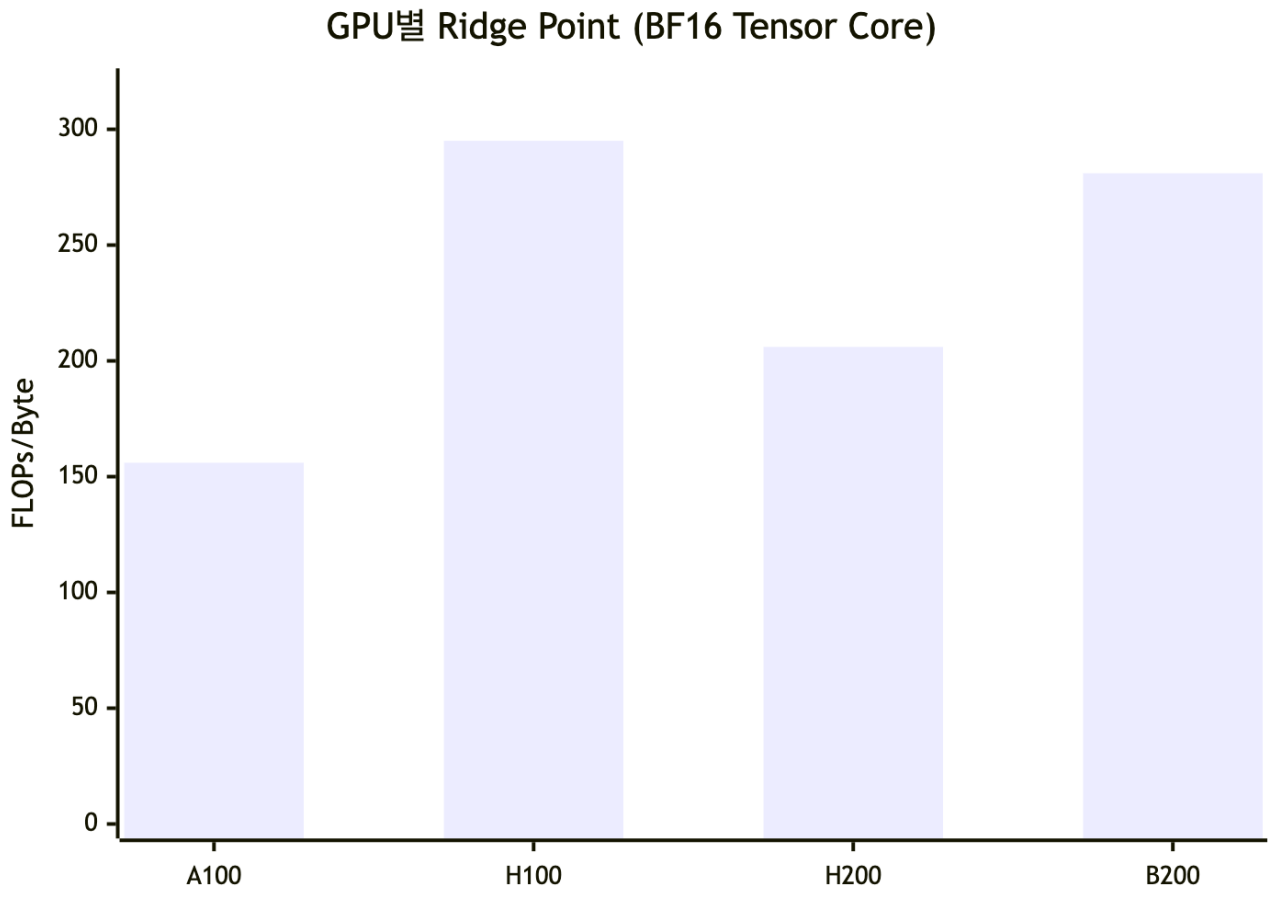
2.4 아키텍처별 AI 값

아키텍처	G	AI (decode)	H100 ridge point 대비
MHA	1	1	0.34%
GQA-4	4	4	1.4%
GQA-8 (Llama 3)	8	8	2.7%
GQA-32	32	32	10.8%
MQA (H_q=128)	128	128	43.4%

핵심 문제: G는 학습 시점에 고정되는 아키텍처 상수이며, 추론 시점에서 조절할 수 없다.

3. GPU 세대별 Ridge Point와 G의 격차

3.1 Ridge Point 추이



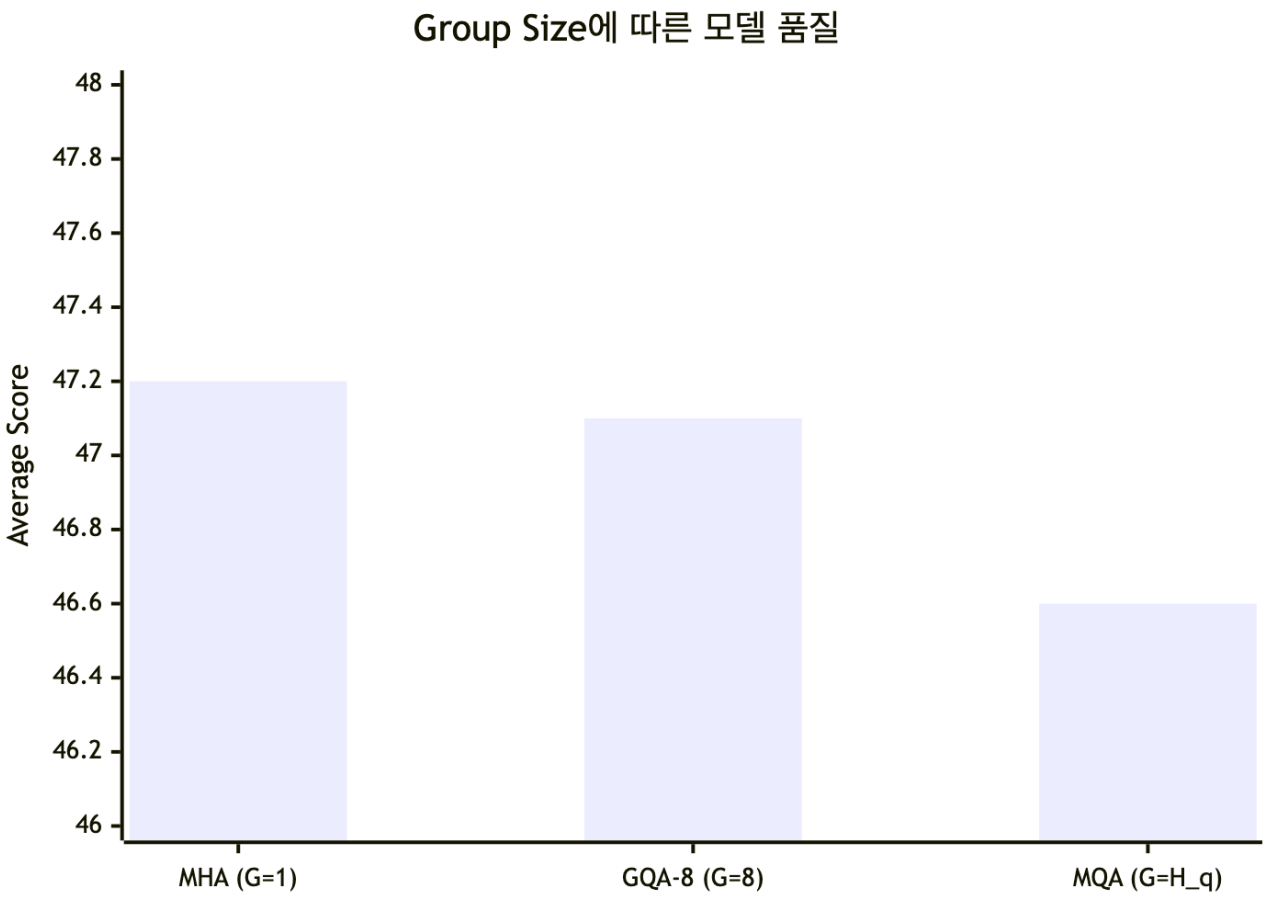
3.2 G=8 (Llama 3급)의 세대별 격차

GPU	Ridge Point	G=8 AI	격차 배율	Tensor Core 유휴율
A100	156	8	19.5×	94.9%
H100	295	8	36.9×	97.3%
H200	206	8	25.8×	96.1%
B200	281	8	35.1×	97.2%

해석: H100에서 Llama-3 70B의 decode attention은 Tensor Core의 2.7%만 활용한다. 나머지 97.3%는 KV cache 로드를 기다리며 놀고 있다. GPU 세대가 발전할수록 연산 성능이 대역폭보다 빠르게 성장하므로, 이 격차는 점점 벌어진다.

3.3 G를 단순히 키우면 안 되는 이유

GQA 원논문(Ainslie et al., 2023)의 품질 비교:



G를 키우면 더 많은 query head가 같은 KV representation을 공유해야 하므로, 각 head가 서로 다른 측면에 attend하는 능력(multi-head diversity)이 감소한다. 이것은 정보 병목(information bottleneck): KV representation이 G개 query head 모두에 유용한 정보를 동시에 인코딩해야 한다.

따라서 GQA로는 AI 문제를 근본적으로 해결할 수 없다.

4. 해결책 1: MLA — Group Size 한계의 근본적 돌파

4.1 핵심 아이디어

DeepSeek-V2/V3의 Multi-Head Latent Attention(MLA)은 G라는 상수 제약을 우회하는 가장 근본적인 해법이다.

발상의 전환: KV head 수를 줄이는 대신, KV 자체를 저차원 latent 벡터로 압축하고, query 쪽에서 projection을 흡수(absorption)하여 사실상 모든 head가 하나의 latent를 공유하면서도 정보 손실을 최소화한다.

4.2 아키텍처 파라미터 (DeepSeek-V3)

파라미터	기호	값
Hidden size	d_emb	7,168
Attention head 수	n_h (= H_q)	128
Head dimension	d_h	128

파라미터	기호	값
RoPE dimension	d_R	64
Compressed KV dimension	d_c	512
전체 decompressed KV 차원	n_h × d_h	16,384

4.3 Absorption의 수학적 유도

Standard MHA Decode (비교 기준)

```
Score_h = q_h × K_h^T                 [1, d_h] × [L, d_h]^T = [1, L]
Output_h = softmax(Score_h) × V_h     [1, L] × [L, d_h] = [1, d_h]
```

각 head h마다 독립적인 K_h, V_h를 HBM에서 로드 → AI = 1

MLA: Latent 압축과 복원

MLA에서 각 토큰의 KV는 저차원 latent c^{KV}로 압축 저장된다:

```
c^{KV} = h × W_DKV        [d_emb] × [d_emb, d_c] → [d_c]     (인코딩, prefill 시)
K_h = c^{KV} × W_UK_h     [d_c] × [d_c, d_h] → [d_h]         (디코딩)
V_h = c^{KV} × W_UV_h     [d_c] × [d_c, d_h] → [d_h]         (디코딩)
```

KV cache에 저장되는 것: c^{KV} (d_c = 512차원) + RoPE key k_R (d_R = 64차원)

→ 토큰당 576 elements vs MHA의 2 × 128 × 128 = 32,768 elements → ~57배 압축

Naive 접근 (Absorption 없이)

```
1) c^{KV}를 K_h, V_h로 복원     ← 이 자체는 GEMM (AI ≈ d_c, 높음)
2) q_h × K_h^T → Score          ← GEMV (AI ≈ 1, 낮음!) ← 여전히 병목
3) Score × V_h → Output         ← GEMV (AI ≈ 1, 낮음!)
```

복원 후에 attention을 하면 expanded된 [L, n_h, d_h] 텐서에 대해 GEMV를 수행하므로, MHA와 동일한 AI ≈ 1 문제가 발생한다.

Absorption (핵심 트릭)

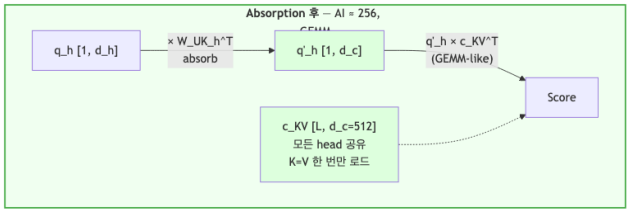
행렬 곱의 결합법칙을 이용하여 W_UK를 Q쪽으로 흡수한다:

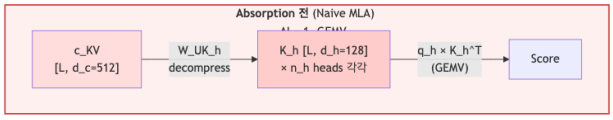
```
Score_h = q_h × K_h^T
        = q_h × (c^{KV} × W_UK_h)^T
        = q_h × W_UK_h^T × (c^{KV})^T
        = (q_h × W_UK_h^T) × (c^{KV})^T      ← 결합 순서 변경!
          ─────────────────────────
          q'_h [1, d_c]      "absorbed query"

Score_h = q'_h × (c^{KV})^T            [1, d_c] × [L, d_c]^T = [1, L]
```

마찬가지로 output 경로도:

```
Output_h = attn_h × V_h
         = attn_h × c^{KV} × W_UV_h
         = (attn_h × c^{KV}) × W_UV_h   ← 결합 순서 변경!
           ──────────────────────────
           o'_h [1, d_c]
```

4.4 Absorption이 만드는 구조적 변화





왜 GEMV가 GEMM이 되는가?

Absorption 후, 128개 query head 모두가 동일한 c^{KV} matrix에 대해 연산한다:

```
Score[h, :] = q'_h × (c^{KV})^T      h = 1, ..., 128
```

이는 사실상:

```
Score = Q' × (c^{KV})^T            [128, d_c] × [L, d_c]^T = [128, L]
```

128개 행을 가진 행렬 × 행렬 = GEMM이다. c^{KV}를 한 번 로드하면 128개 head에 재활용되므로 데이터 재활용률이 G=1에서 G_eff=128로 급등한다.

4.5 MLA Decode의 AI 유도

FLOPs:

Score: 모든 head에 대해 q'_h × (c^{KV})^T

```
FLOPs_score = n_h × 2 × L × d_c
```

Context: attn × c^{KV}

```
FLOPs_context = n_h × 2 × L × d_c
```

합계:

```
FLOPs_total = 4 × n_h × L × d_c
```

Bytes:

K와 V가 동일한 c^{KV}에서 유래하므로 cache를 한 번만 로드:

```
Bytes_KV = L × d_c × 2        (BF16, K와 V에 공유)
Bytes_RoPE = L × d_R × 2      (decoupled RoPE key)
```

FlashMLA에서 c^{KV}를 SRAM에 올려 score와 context 양쪽에 재사용:

```
Bytes_total ≈ L × (d_c + d_R) × 2 = L × 576 × 2
```

AI:

$$\text{AI} = \frac{4 \times n_h \times L \times d_c}{2 \times L \times (d_c + d_R)} \approx 2 \times n_h = 2 \times 128 = 256 \text{ FLOPs/Byte}$$

(d_c >> d_R이고, K·V 공유로 factor 2 발생)

4.6 Ridge Point와 비교

GPU	Ridge Point (BF16 TC)	MLA AI = 256	상태
A100	156	256	Compute-bound
H100	295	256	근접 (87% 활용)
H200	206	256	Compute-bound
B200	281	256	근접 (91% 활용)

MLA는 decode attention을 memory-bound에서 compute-bound로 전환한 최초의 아키텍처이다. GQA-8의 AI=8과 비교하면 32배 개선이며, 이는 G라는 상수 제약을 근본적으로 우회한 결과이다.

4.7 MLA가 품질을 유지하는 이유

GQA/MQA는 KV head를 물리적으로 줄여 G를 키우므로, representation capacity가 감소한다.

MLA는 다르다:

- KV를 저차원으로 압축하되, 복원용 projection W_UK_h, W_UV_h는 head마다 독립적
- 각 head는 고유한 W_UK_h를 통해 latent에서 자기만의 K representation을 추출
- 정보 병목이 아닌 정보 압축: 저차원 latent가 필요한 정보를 보존하도록 학습됨
- 실험 결과 MLA는 MHA와 동등하거나 때로는 더 나은 품질을 달성 (DeepSeek-V2 논문)

5. 해결책 2: 유효 Query 길이를 늘리는 접근

5.1 근거: l_q와 AI의 관계

FlashInfer 공식에서:

AI = 1 / (1/(G × l_q) + 1/L) ≈ G × l_q (G × l_q << L일 때)

Decode의 문제는 l_q = 1이라는 점. l_q를 늘리면 AI가 선형으로 증가한다.

5.2 Speculative Decoding

Speculative decoding은 작은 draft 모델로 k개 토큰을 추측 생성한 뒤, target 모델에서 한 번의 forward pass로 k개를 동시 검증한다:

Draft model: 토큰 t1, t2, t3, t4 생성 (빠름)
Target model: (t1, t2, t3, t4)를 한 번에 검증 → l_q = k = 4

유효 AI 변화:

설정	l_q	AI (GQA-8)	H100 대비
Standard decode	1	8	2.7%
Spec. decode k=4	4	32	10.8%
Spec. decode k=8 (EAGLE-3)	8	64	21.7%
Spec. decode k=16	16	128	43.4%

한계: acceptance rate에 의존하며, draft 모델 자체의 오버헤드가 있다. 또한 $G \times k$가 ridge point를 넘기려면 매우 큰 k가 필요하다 (GQA-8에서 H100 ridge point 295를 넘으려면 $k \approx 37$ 필요).

5.3 Chunked Prefill과 혼합 배치

vLLM 등의 serving 프레임워크에서 prefill request와 decode request를 같은 batch에 혼합(continuous batching)하면:

```
Batch = [Prefill request (l_q = 256)] + [Decode requests × N (l_q = 1 each)]
```

Prefill의 긴 query가 GPU 활용률을 올려주지만, 이는 decode attention 자체의 AI를 개선하는 것이 아니라 시스템 수준의 활용률 개선이다. 개별 decode request의 attention AI는 여전히 G.

6. 해결책 3: KV Tying과 GTA

6.1 GTA (Grouped-Tied Attention)

2025년 제안된 GTA는 GQA 프레임워크 내에서 K = V tying (같은 state를 K와 V에 공유)하는 방식이다:

```
Standard GQA:  KV cache = K [L, H_kv, d_h] + V [L, H_kv, d_h]   (별도 저장)
GTA:           KV cache = S [L, H_kv, d_h]                      (K = V = S, 단일 저장)
```

AI 변화:

$$\text{AI_GTA} = \frac{H_q \times (2 \times L \times d_h)}{(H_{kv} \times L \times d_h) \times 2} = \frac{2 \times H_q}{H_{kv}} = 2G$$

K와 V를 한 번만 로드하므로 FLOPs는 동일하되 Bytes가 절반 → AI가 GQA 대비 2배.

GTA 설정	AI	동등 GQA
GTA-4	8	GQA-8과 같은 AI
GTA-8	16	GQA-16과 같은 AI
GTA-32	64	GQA-64와 같은 AI

의미: GTA-4는 GQA-8과 같은 decode 속도를 내면서, head diversity는 GQA-4 수준을 유지 → 품질/속도 trade-off에서 Pareto 개선.

6.2 한계

GTA도 근본적으로 G에 비례하는 상수이므로, ridge point를 넘기려면 G 자체를 키워야 한다는 한계는 동일하다. MLA 대비 구조적 이점이 제한적.

7. 해결책 4: 양자화로 AI 배수 증가

7.1 원리

양자화는 AI 공식의 분자(FLOPs)를 올리고 분모(Bytes)를 줄여 직접적으로 AI를 배수 증가시킨다:

$$AI_quantized = \frac{Peak_FLOPs(dtype) \times G}{bytes_per_element(dtype)}$$

Data type	Bytes/elem	TC TFLOPS (H100)	AI 배수 (vs BF16)
BF16	2	990	1× (baseline)
FP8	1	1,979	4×
FP4	0.5	3,958	16×

7.2 GQA-8 + 양자화 조합

설정	AI	H100 Ridge Point 대비
GQA-8, BF16	8	2.7%
GQA-8, FP8	32	10.8%
GQA-8, FP4	128	43.4%
MLA, BF16	256	86.8%
MLA, FP8	~1024	Compute-bound (한참 초과)

7.3 SageAttention 시리즈

버전	정밀도	FA2 대비 속도	학회
SageAttention	INT8 QK, FP16 PV	2.1×	ICLR 2025
SageAttention2	INT4 QK, FP8 PV	3×	ICML 2025
SageAttention3	MXFP4	~5×	NeurIPS 2025 Spotlight

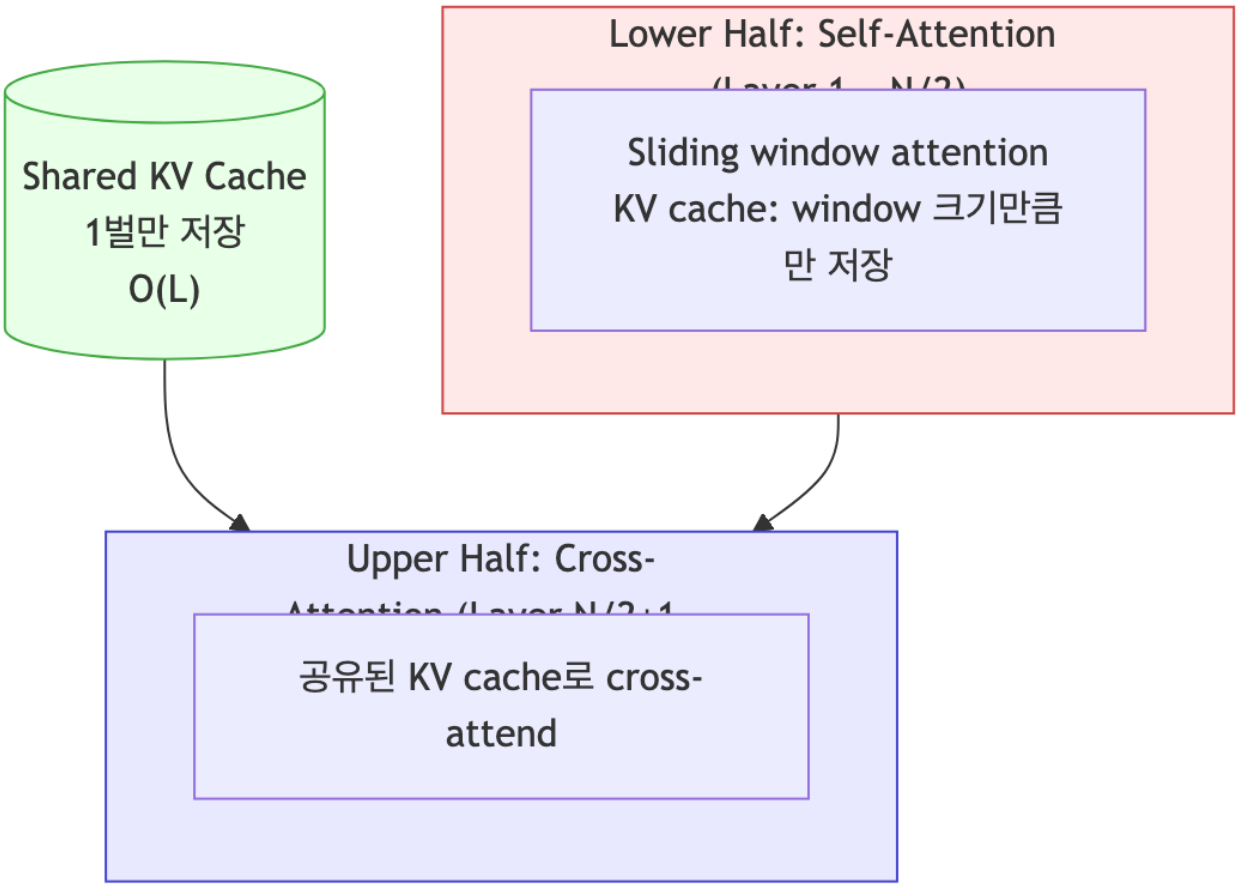
7.4 한계

- 품질 저하 리스크: attention score의 softmax는 outlier에 민감하여, 공격적 양자화 시 품질 하락 가능
- Ridge point도 함께 이동: FP8을 쓰면 peak FLOPs가 2배 올라가므로 ridge point도 변한다. 다만 메모리 대역폭은 데이터 타입에 무관하므로, net effect는 여전히 AI 증가 방향
- 양자화만으로는 MLA 수준의 근본적 전환은 어려움

8. 해결책 5: Cross-Layer KV 공유

8.1 YOCO (You Only Cache Once)

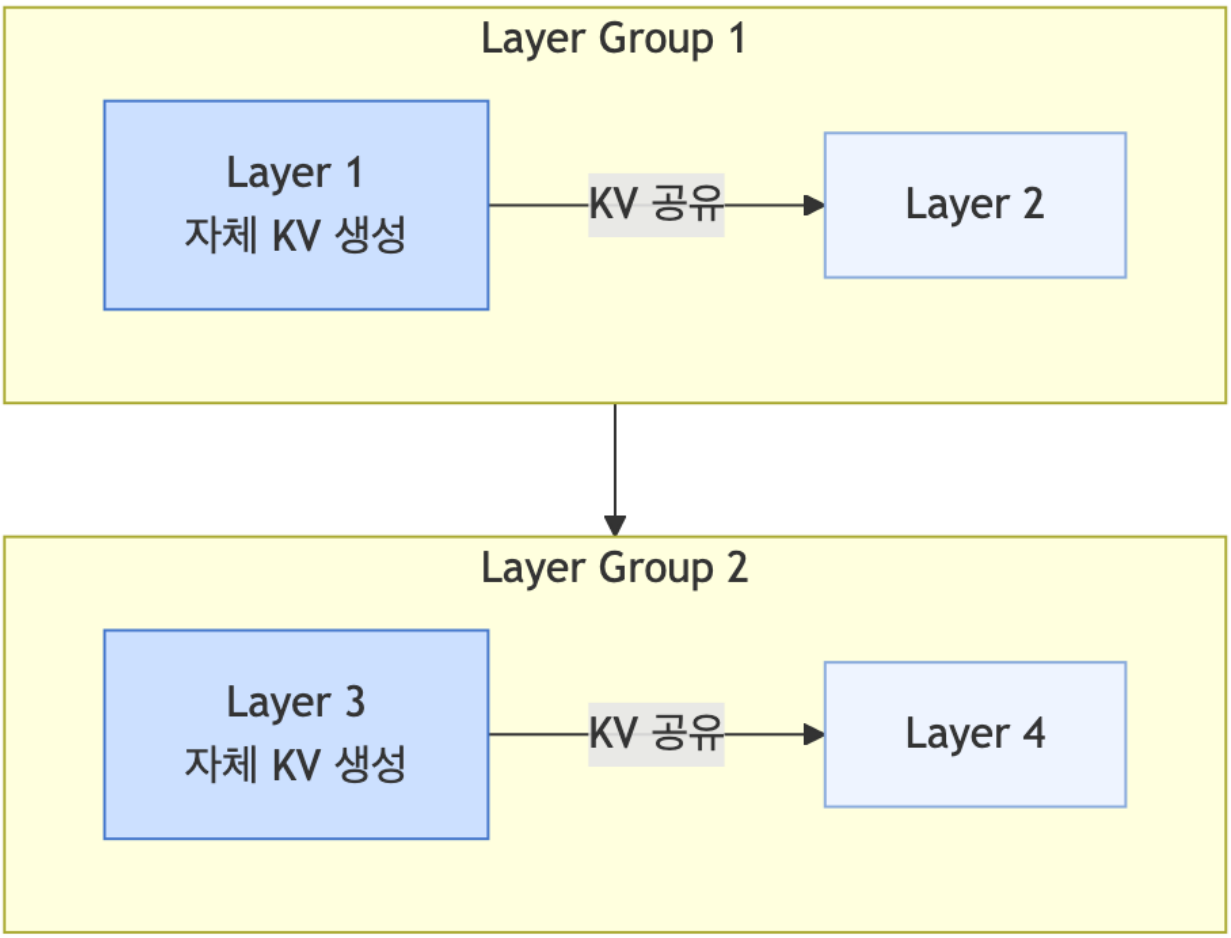
아키텍처를 두 부분으로 나눈다:



- Upper half의 N/2개 layer가 동일한 KV cache를 공유
- KV cache 메모리: O(N × L) → O(L) (N = layer 수)
- Per-layer attention AI는 변하지 않지만, 총 메모리 절약으로 batch size를 키울 수 있어 system-level 처리량이 향상

8.2 CLA (Cross-Layer Attention)

인접 layer들이 KV cache를 공유:



- G_layer = 2이면 KV cache 2배 절감
- 직접적으로 attention AI를 올리지는 않으나, KV 메모리 절약 → 더 큰 batch 가능

8.3 한계

Cross-layer 공유는 시스템 수준 최적화이지, 단일 attention kernel의 AI를 올리는 것이 아니다. Decode attention의 GEMV 본질은 변하지 않는다.

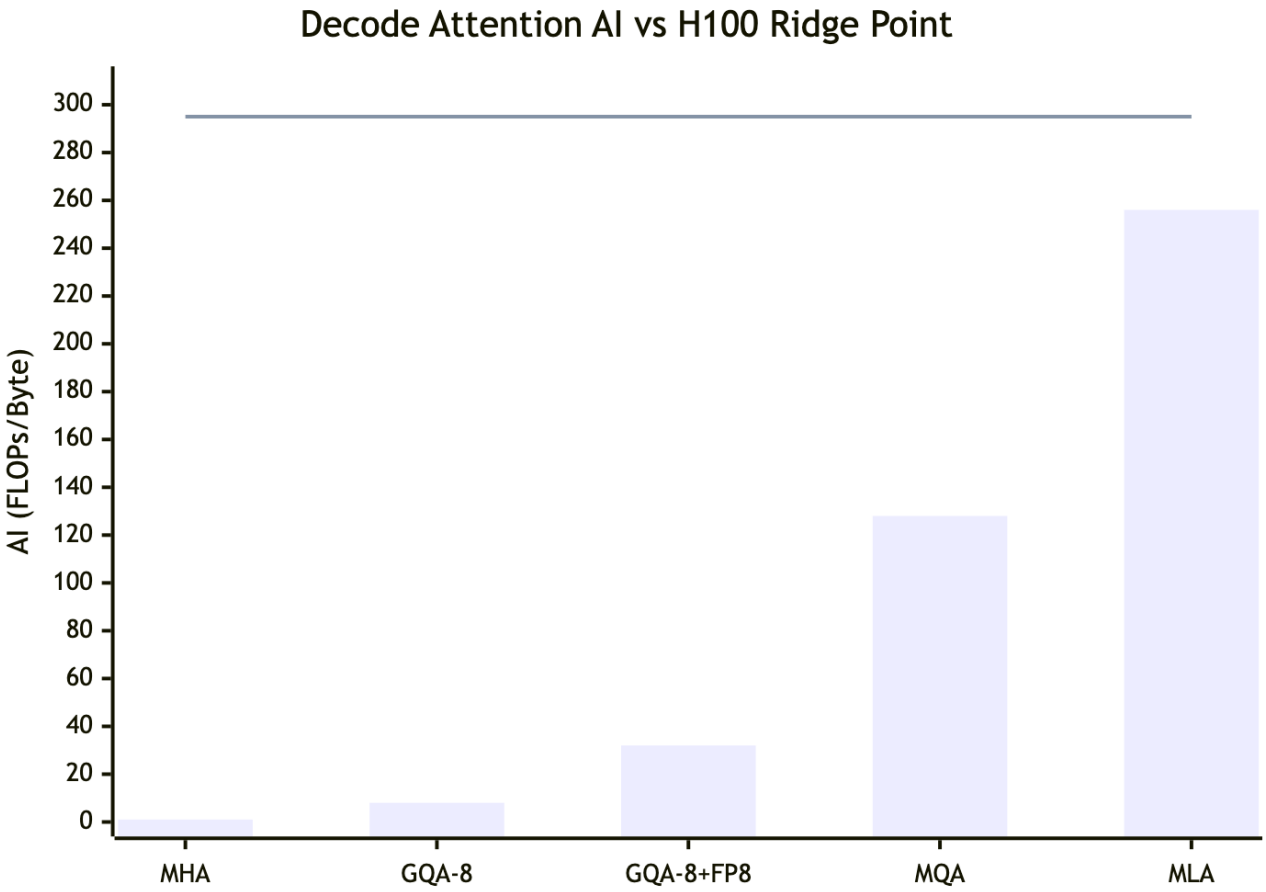
9. 종합 비교와 Roofline 위의 위치

9.1 해결책별 통합 비교

해결책	AI 개선 공식	대표 AI 값	Decode 효과	품질 영향	근본적 해결?
GQA (G 증가)	G	8	O	하락	X
MLA (absorption)	2 × n_h	256	O	유지	O
GTA (K=V tying)	2G	16	O	TBD	X
Speculative decode	G × k	32~64	O	유지	X
FP8 양자화	4× 배수	32 (GQA-8)	O	소폭 하락	X

해결책	AI 개선 공식	대표 AI 값	Decode 효과	품질 영향	근본적 해결?
FP4 양자화	16× 배수	128 (GQA-8)	O	하락 가능	X
Cross-layer 공유	간접적	-	△	소폭 하락	X
FlashAttention tiling	Bytes 감소	~500 (prefill)	X (decode 무효)	유지	X

9.2 Roofline 위의 위치 (H100 BF16 TC 기준)



9.3 해결 전략의 조합 가능성

접근법들은 상호 배타적이지 않다:

```
MLA (AI=256) + FP8 (4×) → AI ≈ 1024  → 완전한 Compute-bound
MLA (AI=256) + Spec. decode (k=2) → AI ≈ 512  → Compute-bound
GQA-8 (AI=8) + FP4 (16×) + Spec. k=4 → AI ≈ 512 → Compute-bound
```

다만 MLA 단독으로도 이미 ridge point 근처이므로, MLA가 가장 효율적인 단일 해결책이다.

10. 결론: 병목의 이동

10.1 2024년의 상황

- Decode attention의 AI = G (상수)
- GPU의 ridge point는 세대마다 증가
- 격차가 벌어지는 구조적 문제 → "개선 불가"라는 비관적 전망

10.2 2025~2026년의 전환

MLA가 게임을 바꿨다:

```
MHA/GQA:  AI = G        (아키텍처 상수, 확장 불가)
MLA:      AI = 2 × n_h  (head 수에 비례, compute-bound 달성)
```

핵심 수학적 트릭은 행렬 곱의 결합법칙을 이용한 연산 순서 재배치(absorption)로, projection matrix를 query 쪽으로 흡수하여:

1. KV cache를 압축 상태 그대로 사용 (Bytes ↓↓)
2. 모든 head가 하나의 latent를 공유 (데이터 재활용 ↑↑)
3. GEMV → GEMM 전환 (Tensor Core 활용 가능)

10.3 새로운 병목

"Rethinking LLM Inference Bottlenecks" (2025~2026)의 결론:

> MLA의 도입으로 attention은 더 이상 LLM 추론의 병목이 아니다. 새로운 병목은:
>
> 1. MoE expert routing: 전문가 선택과 all-to-all 통신
> 2. Interconnect bandwidth: GPU 간 텐서 전송 (NVLink, NVSwitch)
> 3. KV cache 관리: 압축되었지만 여전히 긴 컨텍스트에서 크기가 문제

Attention AI의 상수 한계 문제는 해결되었다. 다음 전쟁은 통신이다.

References

- Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints", EMNLP 2023
- DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model", arXiv:2405.04434
- Shah et al., "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision", NeurIPS 2024
- FlashInfer, "Accelerating Self-Attentions for LLM Serving with FlashInfer", flashinfer.ai
- Lei Mao, "Grouped Query Attention Performance Theoretical Analysis", leimao.github.io
- "Hardware-Efficient Attention for Fast Decoding", arXiv:2505.21487
- "Rethinking LLM Inference Bottlenecks", arXiv:2507.15465
- SageAttention, "SageAttention: Accurate 8-Bit Attention for Plug-and-play Inference Acceleration", ICLR 2025
- "Tiled Flash Linear Attention", arXiv:2503.14376
- "Native Sparse Attention", ACL 2025 Best Paper, arXiv:2502.11089